

SMC CPU Cluster — Low-Level Design

- [SMC CPU Cluster — Low-Level Design & Implementation](#)
 - [Contents](#)
 - [1. Purpose & scope](#)
 - [1.1 Verification scope \(what this IP proves\)](#)
 - [2. SMC top-level architecture \(chiplet view\)](#)
 - [2.1 SMC subsystem role](#)
 - [2.2 Top-level block diagram](#)
 - [2.3 Key parameters](#)
 - [2.4 Memory map \(high-level\)](#)
 - [2.5 Clock & reset domains](#)
 - [2.6 Fabric behavior \(LT abstraction\)](#)
 - [2.7 IP catalog \(units of modeling\)](#)
 - [2.8 Modeling priorities](#)
 - [3. CPU Cluster — role in the SMC](#)
 - [3.1 Position in the chiplet](#)
 - [3.2 External interfaces \(DUT boundary\)](#)
 - [3.3 Internal block view](#)
 - [4. CPU Cluster — implementation](#)
 - [4.1 File layout](#)
 - [4.2 Component inventory](#)
 - [4.3 Spawned processes](#)
 - [4.4 ISS backend \(Whisper\)](#)
 - [4.5 Memory callback bridge & address decode](#)
 - [4.6 IRQ aggregator](#)
 - [4.7 TLM-2.0 sideband — `smc\_axi\_extension`](#)
 - [4.8 CPU Control register file \(BASE + 0x001_0000\)](#)
 - [4.9 Temporal decoupling \(`t1m\_quantumkeeper` \)](#)
 - [4.10 ELF loading](#)
 - [5. Class & component diagrams](#)
 - [5.1 Class diagram \(UML\)](#)
 - [5.2 Component view \(num_harts = 4\)](#)
 - [5.3 Runtime sequence — MMIO load](#)
 - [5.4 Runtime sequence — IRQ wakes a parked hart](#)
 - [6. Build & dependency view](#)
 - [7. Conformance with §3 of 02_SMC_IP_LowLevel_Design](#)
 - [8. Peer-IP integration status](#)
 - [9. References](#)

SMC CPU Cluster — Low-Level Design & Implementation

Companion document to `01_SMC_Architecture.pdf` (chiplet-level architecture) and `02_SMC_IP_LowLevel_Design.pdf` §3 (CPU cluster low-level design). This document is the markdown reference for the SystemC / TLM-2.0 implementation that lives under `libsmc/cpu/`. It is intended to be readable on its own — the PDFs are normative for the *what*, this doc is normative for the *how*.

Contents

1. [Purpose & scope](#)
 2. [SMC top-level architecture \(chiplet view\)](#)
 - 2.1 [SMC subsystem role](#)
 - 2.2 [Top-level block diagram](#)
 - 2.3 [Key parameters](#)
 - 2.4 [Memory map \(high-level\)](#)
 - 2.5 [Clock & reset domains](#)
 - 2.6 [Fabric behavior \(LT abstraction\)](#)
 - 2.7 [IP catalog \(units of modeling\)](#)
 - 2.8 [Modeling priorities](#)
 3. [CPU Cluster — role in the SMC](#)
 - 3.1 [Position in the chiplet](#)
 - 3.2 [External interfaces \(DUT boundary\)](#)
 - 3.3 [Internal block view](#)
 4. [CPU Cluster — implementation](#)
 - 4.1 [File layout](#)
 - 4.2 [Component inventory](#)
 - 4.3 [Spawned processes](#)
 - 4.4 [ISS backend \(Whisper\)](#)
 - 4.5 [Memory callback bridge & address decode](#)
 - 4.6 [IRQ aggregator](#)
 - 4.7 [TLM-2.0 sideband — `smc\_axi\_extension`](#)
 - 4.8 [CPU Control register file \(BASE + 0x001_0000\)](#)
 - 4.9 [Temporal decoupling \(`tlm\_quantumkeeper` \)](#)
 - 4.10 [ELF loading](#)
 5. [Class & component diagrams](#)
 - 5.1 [Class diagram \(UML\)](#)
 - 5.2 [Component view \(num_harts = 4\)](#)
 - 5.3 [Runtime sequence — MMIO load](#)
 - 5.4 [Runtime sequence — IRQ wakes a parked hart](#)
 6. [Build & dependency view](#)
 7. [Conformance with §3 of 02_SMC_IP_LowLevel_Design](#)
 8. [Peer-IP integration status](#)
 9. [References](#)
-

1. Purpose & scope

The **System Management Controller (SMC)** is the per-chiplet subsystem that owns clock / voltage / reset management, hardware platform bring-up, security filtering, interrupt aggregation and inter-chiplet communication. It is built around a small firmware-driven **RV64GC microcontroller cluster** of 1–4 cores.

This document covers two layers:

1. **Chiplet view (§2)** — the SMC's role in the OCAH SMU, its TLM topology, memory map, and the catalogue of peer IPs that the cluster interacts with. This is a summary of `01_SMC_Architecture.pdf` and is provided so a reader of the cluster code has the surrounding context in one searchable place.
2. **CPU cluster view (§3 – §7)** — the architecture and implementation of `smc::smc_cpu_cluster`, the SystemC/TLM-2.0 model under `libsmc/cpu/`. This is the deliverable that the markdown documents in this directory (`03_CPU_Cluster_Test_Plan.md`, `04_CCI_Integration_Guide.md`) verify.

The implementation uses the Tenstorrent **Whisper** ISS (`WdRiscv::System<uint64_t>` / `WdRiscv::Hart<uint64_t>`) as the sole execution engine, behind a backend-agnostic `iss_hart` interface so the cluster does not call Whisper APIs directly.

1.1 Verification scope (what this IP proves)

`03_CPU_Cluster_Test_Plan.md` and `test/cluster_tb` verify the **cluster model** and **integration hooks** to peer IPs. They do not replace peripheral-IP or SoC-level plans.

Aspect	Owner	<code>cluster_tb</code>
PLIC IP functional matrix	<code>peripherals/plic</code> + <code>plic_tb</code>	Real <code>plic.cpp</code> linked; exhaustive matrix in <code>plic_tb</code>
Cluster ↔ PLIC (MMIO + <code>irq_ext</code> + ISR)	§8.7 of test plan	In scope
CLINT IP (mtime, MSIP MMIO, ...)	<code>peripherals/clint</code> (future)	Out of scope — <code>irq_sw</code> / <code>irq_timer</code> stubs only (§8.6)
Scratchpad on <code>data @ 0xC006_0000</code>	Fabric router (future)	<code>INIT_MEM_DONE</code> handshake via <code>ScratchpadSramStub</code> only
<code>riscv_plic0.c</code> / production IRQ map	Firmware / SoC test plan	Open

See also `03_SMC_Test_Plan.md` §A.2 for the repo-wide GTest vs `*_tb` patterns.

2. SMC top-level architecture (chiplet view)

2.1 SMC subsystem role

Source: `01_SMC_Architecture.pdf` §1.

The SMC is a per-chiplet subsystem providing:

- Clock, voltage and reset management (PLLs, reset trees, FLR, AVS).
- Hardware platform configuration and bring-up (boot ROM, eFuse, straps).

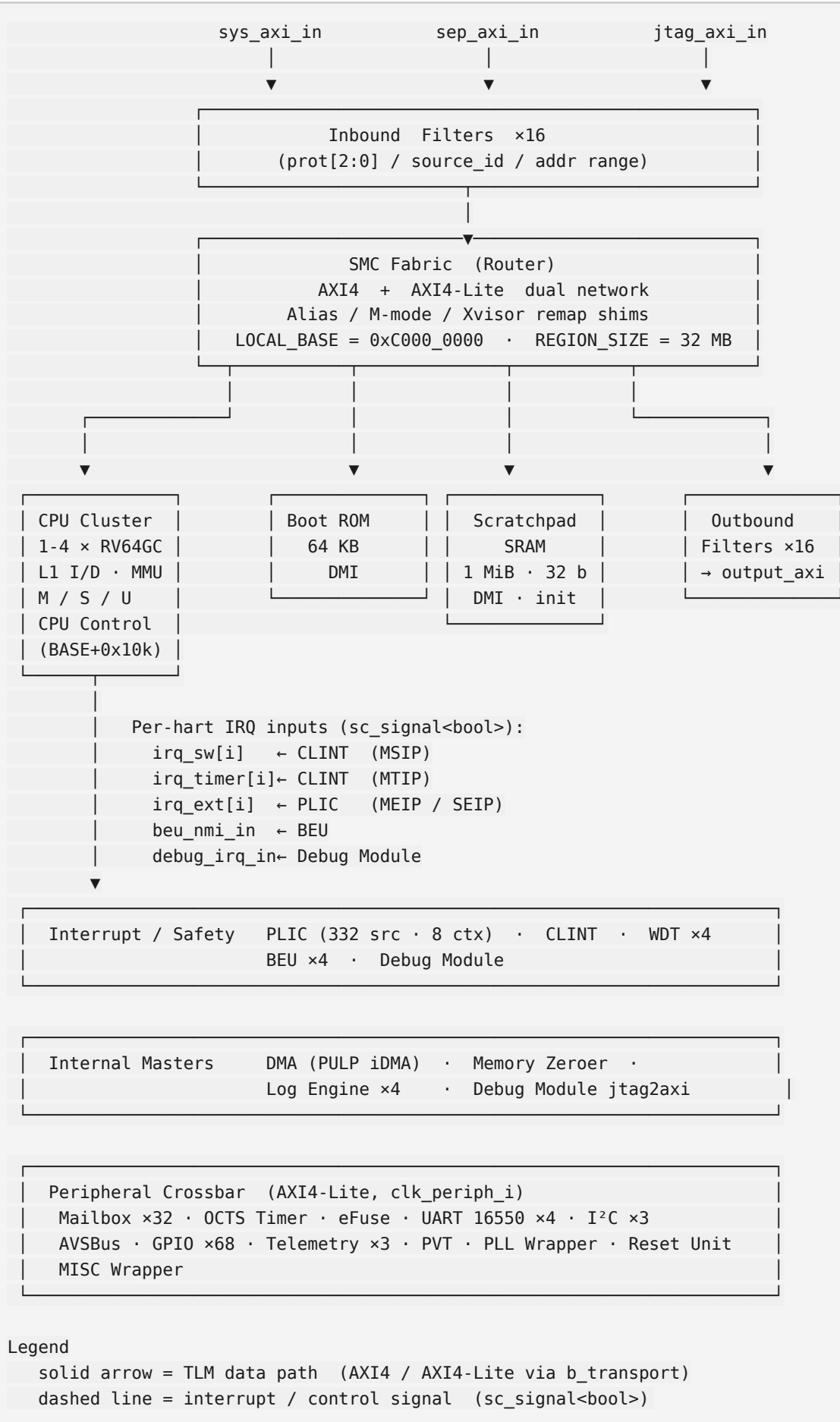
- System monitoring & telemetry (PVT, ATB telemetry, log engine).
- Inter-processor / inter-chiplet communication (mailboxes, OCTS time sync, I²C/I³C).
- Security-aware fabric (address remap, inbound/outbound filters, protection bits).
- Centralised interrupt management (PLIC, CLINT, per-core watchdogs, BEUs).

In a multi-chiplet SiP each chiplet contains one SMC; the SMC of the primary chiplet additionally orchestrates SiP-level tasks for secondary chiplets via the OCCP protocol.

All control logic lives in **firmware** running on the cluster; HW IPs are added only when firmware cannot meet performance / determinism / security requirements.

2.2 Top-level block diagram

Source: 01_SMC_Architecture.pdf §2, re-drawn for the SystemC/TLM-2.0 modeling unit boundary.



Each block in the figure above corresponds to **one SystemC modeling unit** listed in §2.7.

2.3 Key parameters

Source: 01_SMC_Architecture.pdf §2.1.

Parameter	Value
CPU cores	1–4 RV64GC
Aperture size	32 MB at SMC-local base 0xC000_0000
AXI4 data width	64-bit
AXI4 system address width	56-bit
AXI4 internal address width	32-bit
Maximum interrupt sources at PLIC	332 (326 active)
Peripheral clock domain	≥ 100 MHz

2.4 Memory map (high-level)

Source: 01_SMC_Architecture.pdf §2.2. All offsets are relative to LOCAL_BASE = 0xC000_0000 (read-only) which aliases to a firmware-programmable GLOBAL_BASE (default 0x4000_0000).

Range (BASE+)	Size	Block
0x000_0000 – 0x000_0FFF	4 KB	Watchdog Timers (4 cores)
0x000_1000 – 0x000_1FFF	4 KB	RISC-V Debug Module
0x000_2000 – 0x000_3FFF	8 KB	Reset Unit, MISC, PLL Wrapper
0x000_4000 – 0x000_4FFF	4 KB	GPIO Interface (68) + GPIO Control
0x000_7000 – 0x000_AFFF	16 KB	PVT, AVSBus, I ² C, UART
0x000_B000 – 0x000_FFFF	20 KB	eFuse, Telemetry, OCTS, DTP/DFT
0x001_0000 – 0x001_7FFF	32 KB	CPU Control , Alias / M-mode / Xvisor remap, Filters
0x001_8000 – 0x003_7FFF	128 KB	Mailbox (32 channels)
0x003_8000 – 0x003_83FF	1 KB	DMA + Zeroer
0x004_0000 – 0x007_FFFF	256 KB	ROM + Scratchpad SRAM
0x100_0000 – 0x17F_FFFF	8 MB	M-mode remapped region
0x180_0000 – 0x1FF_FFFF	8 MB	Xvisor remapped region
0x400_0000 – 0x43F_FFFF	4 MB	PLIC
0x800_0000 – 0x800_FFFF	64 KB	CLINT
0x801_0000 + N×0x1000	4×4 KB	Bus Error Units

The **CPU Control** sub-window (32 KB at BASE + 0x001_0000) is owned by the cluster's ctrl target socket (see §4.8).

2.5 Clock & reset domains

Source: 01_SMC_Architecture.pdf §3.

Domain	Drives
<code>clk_smc_i</code>	CPU cluster, AXI4 fabric, alias remap, filters
<code>clk_ref_i</code>	CLINT, OCTS, debug, reset CDC anchor, PLL ref
<code>clk_periph_i</code>	AVSBus, I ² C, UART, I ³ C, peripheral crossbar
<code>clk_telemetry_i</code>	Telemetry receiver (sourced by ATB)

Reset hierarchy:

- **Cold Reset** (POR / external) — primary reset.
- **Cool Reset** (BMC / FLR sequence) — primary reset, with isolation.
- **Core Reset** — CPU + private caches, PLIC, CLINT, WDT, BEU.
- **Function Level Reset (FLR)** — PCIe-driven; programmable pre-reset delay (`ISOLATE_REQ_FLR_COUNTER_VALUE`) and reset hold (`ISOLATE_REQ_FLR_RESET_COUNTER_VALUE`); auto-bypasses memory repair via `skip_mem_repair_o`.
- **Watchdog Reset / Debug Reset** — core-local.

LT-modeling implications:

- Clocks are abstract; no waveform is generated. Each module exposes a single `sc_event`-based reset and uses `sc_time` deltas for delay annotations.
- Reset signals are modelled as boolean inputs that trigger `SC_METHOD` initialisers.

2.6 Fabric behavior (LT abstraction)

Source: `01_SMC_Architecture.pdf` §4.

The SMC fabric is a **dual-network interconnect**:

- AXI4 high-performance network — CPU cluster, SRAM, DMA, filtering, external AXI ports.
- AXI4-Lite low-performance network — peripherals and configuration registers.

In LT, the fabric is one **TLM-2.0 router module** that exposes:

- one `tlm_utils::simple_target_socket` per IP;
- one `tlm_utils::simple_initiator_socket` per master (CPU, DMA, JTAG2AXI, Log Engine, `sys_axi_in`, `sep_axi_in`).

Every transaction carries a `tlm::tlm_generic_payload` extended with the sideband fields described in §4.7 (`smc::smc_axi_extension`). Address remapping (alias / M-mode / Xvisor) and the inbound / outbound filters are **purely functional** in LT — latency is annotated only when needed for performance studies (default 0 ns).

2.7 IP catalog (units of modeling)

Source: `01_SMC_Architecture.pdf` §5. Each entry is one SystemC modeling unit. Per-IP detail is in `02_SMC_IP_LowLevel_Design.pdf`.

#	IP module	TLM role	Bus	Approx. reg space
1	smc_cpu_cluster (RV64GC wrapper)	Initiator + Target	AXI4 / AXI4-Lite	8 KB (CPU Ctrl)
2	plic (332 sources)	Target	AXI4-Lite	4 MB
3	clint (4 cores)	Target	AXI4-Lite	64 KB
4	wdt (per-core watchdog ×4)	Target	APB4	1 KB each
5	bus_error_unit (per-core BEU ×4)	Target	AXI4-Lite	4 KB each
6	boot_rom	Target	AXI4-Lite	up to 64 KB
7	scratchpad_sram (32 banks, 1 MiB)	Target	AXI4	128 KB × 2 windows
8	smc_fabric (router + remap)	Router	AXI4 + AXI4-Lite	—
9	axi_filter (inbound ×16 + outbound ×16)	Filter	AXI4 / AXI4-Lite	32 B per filter
10	dma_engine (PULP iDMA)	Initiator + Target	AXI4 + AXI4-Lite	512 B
11	memory_zeroer	Initiator + Target	AXI4 + AXI4-Lite	512 B
12	mailbox_unit (up to 32 pairs)	Target	AXI4-Lite	2 KB × 64
13	system_timer_octs	Target + pulse I/F	AXI4-Lite / APB4	36 B
14	efuse (ctrl + shadow + bank + SHIM)	Target	APB4 + AXI4-Lite	4 KB total
15	uart_16550 (×4)	Target	AXI4-Lite	64 B
16	log_engine (×4)	Initiator + Target	AXI4-Lite	128 B
17	i2c_controller (×3, OpenTitan-style)	Target + initiator-on-bus	AXI4-Lite	4 KB
18	avsbus_controller	Target	AXI4-Lite / APB4	256 B
19	gpio (68 + ctrl)	Target	AXI4-Lite	16 B/inst + 32 B ctrl
20	telemetry_receiver (×3, ATB)	Target + ATB sink	AXI4-Lite	128 B
21	reset_unit	Target	AXI4-Lite	2 KB
22	pll_wrapper	Target	AXI4-Lite	4 KB
23	misc_wrapper	Target	AXI4-Lite	2 KB
24	debug_module (RISC-V DM)	Target + initiator (jtag2axi)	AXI4-Lite + APB	4 KB
25	pvt_wrapper	Target	AXI4-Lite	4 KB

I³C controllers (§6.10) are listed in the spec as RESERVED / NOT YET IMPLEMENTED and are therefore excluded from the initial modeling scope.

2.8 Modeling priorities

Source: 01_SMC_Architecture.pdf §5.2.

Priority	Modules	Rationale
P0	<code>smc_fabric</code> , <code>axi_filter</code> , <code>smc_cpu_cluster</code> (stub), <code>pllc</code> , <code>clint</code> , <code>boot_rom</code> , <code>scratchpad_sram</code> , <code>reset_unit</code> , <code>misc_wrapper</code> , <code>mailbox_unit</code> , <code>system_timer_octs</code> , <code>uart_16550</code> , <code>gpio</code> , <code>efuse</code>	Required to boot SMC firmware up to OCCP loop.
P1	<code>dma_engine</code> , <code>memory_zeroer</code> , <code>log_engine</code> , <code>i2c_controller</code> , <code>avsbu_controller</code> , <code>pvt_wrapper</code> , <code>telemetry_receiver</code> , <code>pll_wrapper</code> , <code>wdt</code> , <code>bus_error_unit</code>	Required for full functional system management workloads.
P2	<code>debug_module</code>	Needed only when modelling JTAG / debug flows.

3. CPU Cluster — role in the SMC

3.1 Position in the chiplet

The cluster is **the only firmware-executing IP in the SMC**. Every management decision (boot, FLR, telemetry, OCCP, AVS, security) is taken by software running on 1–4 RV64GC harts inside it; every other SMC IP exists to give that firmware deterministic, low-latency hooks into the hardware.

The cluster is wired to the SMC fabric as both an **AXI initiator** (for hart loads/stores and instruction fetches into the fabric) and an **AXI-Lite target** (for the 8 KB CPU Control register block at BASE + `0x001_0000`).

3.2 External interfaces (DUT boundary)

The cluster's only contact points with the rest of the chiplet are:

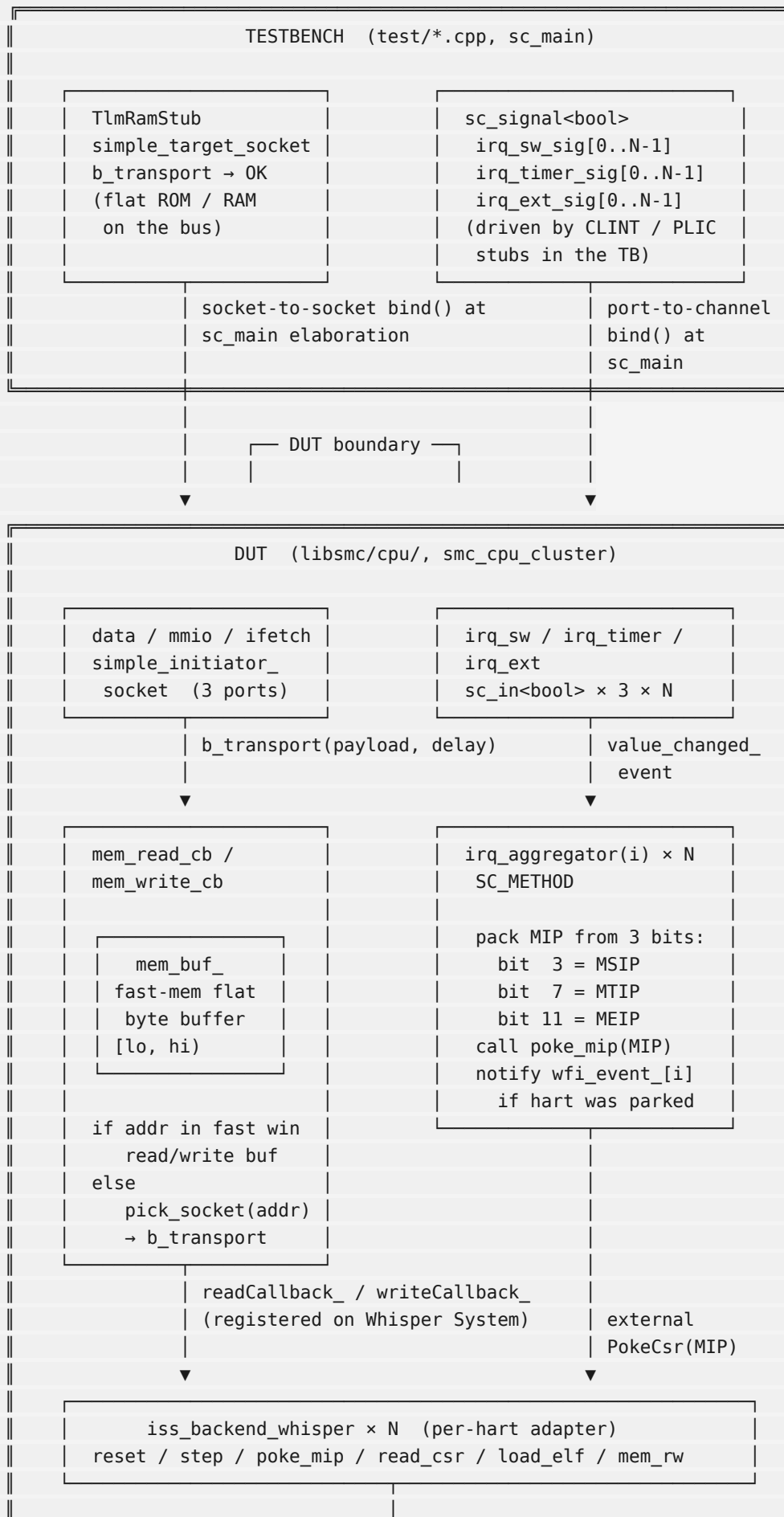
Direction	Object	Type	Purpose
out → fabric	<code>data</code>	<code>simple_initiator_socket<...,64></code>	High-perf data accesses (SRAM, DMA target, internal SRAM)
out → fabric	<code>mmio</code>	<code>simple_initiator_socket<...,64></code>	Peripheral / register-window accesses (PLIC, CLINT, peripherals)
out → fabric	<code>ifetch</code>	<code>simple_initiator_socket<...,64></code>	Instruction fetch path (currently routed identically; reserved for an ICache decode hook)
in ← fabric	<code>ctrl</code>	<code>simple_target_socket<...,64></code>	CPU Control register file (RESET_VECTOR_N, CORE_ENABLE, LOCAL/GLOBAL_BASE, REGION_SIZE, ...)
in ← CLINT	<code>irq_sw[i]</code>	<code>sc_in<bool></code>	Per-hart software interrupt (MSIP / MIP[3])
in ← CLINT	<code>irq_timer[i]</code>	<code>sc_in<bool></code>	Per-hart timer interrupt (MTIP / MIP[7])
in ← PLIC	<code>irq_ext[i]</code>	<code>sc_in<bool></code>	Per-hart external interrupt (MEIP / MIP[11])

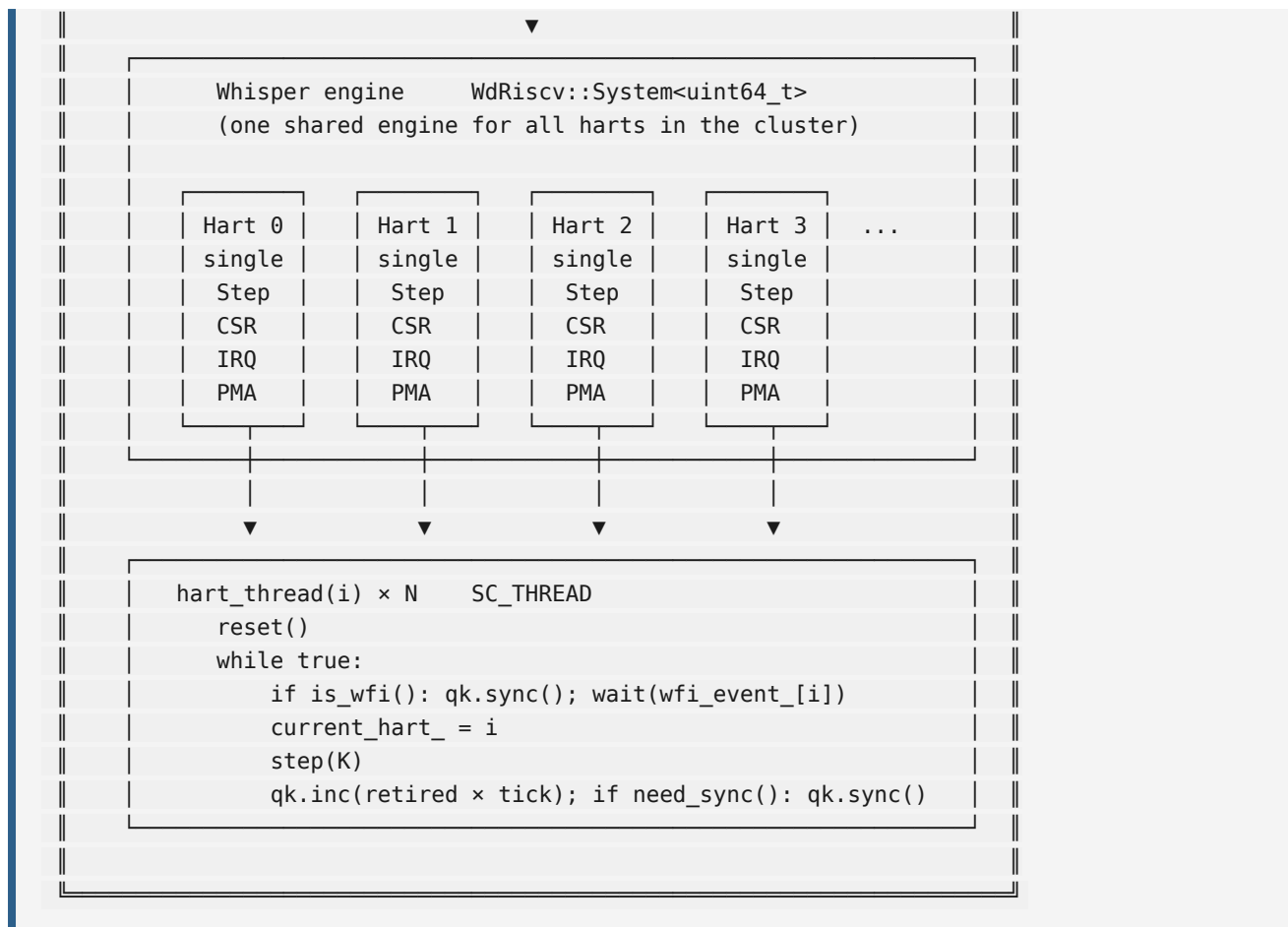
Future / not-yet-wired (gated on peer IPs):

Direction	Object	Source
in ← BEU	<code>beu_nmi_in</code>	Bus Error Unit
in ← Debug	<code>debug_irq_in</code>	Debug Module
in ← PLIC	<code>irq_ext_s[i]</code>	PLIC (S-mode context) — see <code>03_CPU_Cluster_Test_Plan.md</code> §13.1

3.3 Internal block view

Plain-ASCII “read this first” sketch — renders in any editor, with the DUT / TB split made explicit:





The right-hand side shows how a `sc_signal<bool>` rising edge becomes a MIP poke and a WFI wake-up; the left-hand side shows how a hart load becomes a `b_transport` on one of three initiator sockets after the fast-mem fast path is consulted. All four sockets and the IRQ inputs are the cluster's only external interfaces — every other peer IP plugs into one of them.

4. CPU Cluster — implementation

4.1 File layout

```
external/
  whisper/                ← git submodule (tenstorrent/whisper, pinned SHA)
  whisper-cmake/
    CMakeLists.txt        ← build shim (ExternalProject wrapping GNUmakefile)

libsmc/cpu/
  iss_hart.h              ← pure abstract C++ interface (backend-agnostic)
  iss_bus_bridge.h        ← TLM bridge helpers (pick_socket, ext setup)
  iss_backend_whisper.h   ← Whisper-specific concrete adapter
  iss_backend_whisper.cpp ← sole ISS backend
  smc_cpu_cluster.h       ← sc_module declaration
  smc_cpu_cluster.cpp     ← processes, IRQ aggregator, CPU Control regs
  smc_axi_extension.h     ← canonical sideband tlm_extension
  CMakeLists.txt          ← libsmc_cpu_cluster.a target

CMakeLists.txt (cluster top-level)
  ← finds SystemC / Whisper / Boost / CCI;
  ← optionally builds the PLIC integration
  ← via SMC_BUILD_PLIC_INTEGRATION (default ON).

test/                    ← see 03_CPU_Cluster_Test_Plan.md
```

4.2 Component inventory

Owned object	Type	Count	Purpose
whisper_sys_	unique_ptr<WdRiscv::System<uint64_t>>	1	Multi-hart Whisper engine
harts_[i]	unique_ptr<iss_backend_whisper>	N	Per-hart adapter to abstract iss_hart
qk_[i]	tlm_quantumkeeper	N	Local time per hart (temporal decoupling)
wfi_event_[i]	sc_event	N	WFI park / wake-up signal
core_enable_event_[i]	sc_event	N	CORE_ENABLE-driven park / re-enable
mem_buf_	vector<uint8_t>	1	Fast-memory window for non-MMIO addresses
current_hart_	unsigned	1	Hart index visible to the read/write callbacks
data, mmio, ifetch	simple_initiator_socket<...,64>	3	TLM masters to the SMC fabric
ctrl	simple_target_socket<...,64>	1	CPU Control register file target
irq_*[i]	sc_vector<sc_in<bool>>	3 × N	M-mode software / timer / external IRQ inputs

4.3 Spawned processes

Process	Kind	Sensitivity	Body
<code>hart_thread(i)</code>	SC_THREAD	none (runs at <code>sc_start</code>)	reset → <code>step(K)</code> loop with <code>qk + WFI / CORE_ENABLE</code> park
<code>irq_aggregator(i)</code>	SC_METHOD	<code>irq_sw[i]</code> , <code>irq_timer[i]</code> , <code>irq_ext[i]</code>	pack MIP → <code>poke_mip()</code> → notify <code>wfi_event_[i]</code> if hart was parked
<code>ctrl_b_transport</code>	callback	invoked by <code>ctrl</code> target socket	32 / 64-bit decode against the CPU Control register file (§4.8)

4.4 ISS backend (Whisper)

The cluster never calls Whisper APIs directly. All execution goes through the pure abstract `iss_hart` interface:

Method	Purpose
<code>reset()</code>	Reset hart; PC goes to address set by <code>set_reset_pc()</code>
<code>step(K)</code>	Execute up to <i>K</i> instructions (default loops one at a time)
<code>set_reset_pc(addr)</code>	Set boot vector (takes effect on the next <code>reset()</code>)
<code>get_pc()</code>	Read current program counter
<code>poke_mip(val)</code>	Inject machine interrupt-pending bits
<code>read_csr(num)</code>	Read any CSR by number
<code>is_wfi()</code>	True if the hart is stalled in WFI
<code>clear_wfi()</code>	Clear the WFI stall flag (called after poking MIP)
<code>inject_nmi(cause)</code>	Schedule an NMI (<code>Hart::setPendingNmi()</code>); wakes a parked hart
<code>last_commit()</code>	{ <code>pc</code> , <code>opcode</code> , <code>trapped</code> , <code>trap_cause</code> , <code>was_wfi</code> , <code>priv</code> } snapshot
<code>current_priv()</code>	Live <code>Hart::privilegeMode()</code> (0 = U, 1 = S, 3 = M)
<code>load_elf(paths)</code>	<code>System::loadElfFiles({elf}, raw=false, verbose=false)</code>
<code>mem_read / mem_write</code>	Side-effect-free debug accesses (for testbench / Debug Module)

The Whisper-specific concrete class `iss_backend_whisper` maps each of these to the underlying engine:

iss_hart method	Whisper API used
reset()	Hart::reset()
step(K)	for (i<K) hart->singleStep(di_) (loop)
set_reset_pc(v)	Hart::defineResetPc(v)
poke_mip(v)	Hart::externalPokeCsr(CsrNumber::MIP, v, false)
read_csr(n)	Hart::peekCsr(CsrNumber(n))
is_wfi()	internal wfi_active_ flag, set when di_.instId() == InstId::wfi
clear_wfi()	implicit — poke_mip(v != 0) clears wfi_active_
Memory read	System::defineReadMemoryCallback(fn)
Memory write	System::defineWriteMemoryCallback(fn)
Load ELF	System::loadElfFiles({elf}, false, false)

WFI detection. After each `singleStep(di_)`, the backend checks `di_.instId() == InstId::wfi`. If true and `(MIP & MIE) == 0`, it sets `wfi_active_ = true` and breaks the inner step loop. Whisper's default `wfiTimeout_ = 1` advances the PC past the WFI immediately, which is why the decoded-instruction check is used in preference to a PC-stall heuristic.

WFI-wake quirk. After an external MIP poke wakes a parked hart, Whisper may dispatch to MTVEC on the next step even with `MSTATUS.MIE = 0` and `MIE = 0`. The tests install a `j . self-loop` at `MTVEC = 0` in fast-mem so Whisper lands on a valid instruction; the wrapper itself is unaffected (the wake event is delivered correctly).

4.5 Memory callback bridge & address decode

Whisper's `MEM_CALLBACKS=1` build mode lets the cluster intercept every load and store through two C++ callbacks registered on the `WdRiscv::System` at construction time:

```
sys_.defineReadMemoryCallback ([this](uint64_t a, unsigned s, uint64_t& d) {
    return mem_read_cb(a, s, d);
});
sys_.defineWriteMemoryCallback([this](uint64_t a, unsigned s, uint64_t d) {
    return mem_write_cb(a, s, d);
});
```

The bridge logic in each callback is:

- Fast path** — if `addr ∈ [fast_mem_lo, fast_mem_hi)`, copy from / to `mem_buf_`. No TLM activity.
- Slow path** — `pick_socket(addr)` selects one of `data`, `mmio`, `ifetch`:
 - `mmio` for addresses inside the `[cfg.mmio_lo, cfg.mmio_hi)` window (peripherals, CPU Control, PLIC, CLINT, ...);
 - `ifetch` for instruction-fetch accesses (currently routed identically to `data`; reserved for an ICache decode hook);
 - `data` for everything else (SRAM, DMA targets, etc.).
- Build a `tlm_generic_payload`, attach an `smc_axi_extension` (§4.7), call `b_transport(trans, delay)` on the picked socket, and sync the per-hart `tlm_quantumkeeper` with the returned `delay`.
- Data packing.** Whisper passes data as a packed `uint64_t`, not a byte buffer. The bridge `memcpy-packs / -unpacks` via a stack-local `uint8_t buf[8]`.

```

bool mem_rd(uint64_t addr, unsigned size, uint64_t& data) {
    uint8_t buf[8] = {};
    tlm::tlm_generic_payload trans;
    trans.set_read();
    trans.set_address(addr);
    trans.set_data_ptr(buf);
    trans.set_data_length(size);
    set_axi_ext(trans, /*source_id*/cfg_.source_id,
                /*priv*/priv_from_hart(),
                /*axi_id*/current_hart_);
    sc_time delay = qk_->get_local_time();
    pick_socket(addr)->b_transport(trans, delay);
    qk_->set(delay);
    std::memcpy(&data, buf, size); // little-endian
    return trans.is_response_ok();
}

```

4.6 IRQ aggregator

One `SC_METHOD` per hart, sensitive to the three IRQ input ports:

```

irq_aggregator(i):
    mip = 0
    mip |= (irq_sw [i].read() << 3) // MSIP
    mip |= (irq_timer[i].read() << 7) // MTIP
    mip |= (irq_ext [i].read() << 11) // MEIP
    parked_before = harts_[i]->is_wfi() // snapshot BEFORE the poke
    harts_[i]->poke_mip(mip) // externalPokeCsr(MIP, mip)
    if mip != 0 and parked_before:
        wfi_event_[i].notify()

```

Ordering invariant. `poke_mip(non-zero)` clears `wfi_active_` as a side effect, so the `is_wfi()` snapshot must be taken **before** the poke. If snapped after, every wake-from-WFI path silently drops the `notify()` and the parked hart never resumes. This is the single most common regression in this area; the test `cluster_irq_test` (TC-CPU-007/008/009) pins it.

4.7 TLM-2.0 sideband — `smc_axi_extension`

Canonical definition: `libsmc/cpu/smc_axi_extension.h`.

Every outgoing transaction carries a `tlm::tlm_extension` describing the master and access type. The class is defined under the `smc` namespace and is **shared verbatim** with every other SMC IP that needs to read or write it (e.g. the PLIC; the cluster's header is included unchanged under the same name in `peripherals/plic/include/smc_tlm_extensions.h` via an `__has_include` shim, so a single binary that links both modules sees one and only one `smc::smc_axi_extension` type — the ODR contract is held).

Field	Type	Default	Meaning
<code>source_id</code>	<code>uint16_t</code>	<code>SMC_CPU_SOURCE_ID</code>	Originating master ID (cluster default = <code>0x10</code>)
<code>axi_id</code>	<code>uint16_t</code>	<code>0</code>	AXI transaction ID; cluster sets to the hart index
<code>prot</code>	<code>uint8_t</code>	<code>0b0111</code>	AXI4 PROT[3:0] (bit assignments below)
<code>is_locked</code>	<code>bool</code>	<code>false</code>	Mirror of <code>prot[3]</code> ; set for AMO / LR-SC
<code>is_fetch</code>	<code>bool</code>	<code>false</code>	Mirror of <code>!prot[0]</code> ; set for instruction-fetch transactions
<code>is_secure</code>	<code>bool</code>	<code>false</code>	Mirror of <code>!prot[1]</code> ; secure mode (not modelled yet)
<code>is_user</code>	<code>bool</code>	<code>false</code>	Mirror of <code>!prot[2]</code> ; tracks U-mode vs M/S-mode

`prot` bit assignments:

```

bit [0] : data (1) vs instruction (0)
bit [1] : non-secure (1) vs secure (0)
bit [2] : privileged (M/S = 1) vs user (U = 0)
bit [3] : locked (atomic AMO / LR-SC) – 1 when in a locked sequence

```

Population rules in `tlm_access()` (cluster source):

- `source_id = cfg_.source_id` (default `0x10`).
- `axi_id = current_hart_`.
- `is_user ← iss_hart::current_priv() == 0`.
- `is_locked ← cfg_.amo_lock_detect && last_commit().opcode[6:0] == 0x2F` (catches both AMO and LR/SC).

4.8 CPU Control register file (BASE + 0x001_0000)

Mapped per `01_SMC_Architecture.pdf` §2.2 and `02_SMC_IP_LowLevel_Design.pdf` §3.8.

The cluster owns a TLM-2.0 target socket `ctrl` that decodes a small register window. Offsets and widths:

Offset	Width	Reg	Notes
0x000	64	RESET_VECTOR_0	Write also calls <code>harts_[0]->set_reset_pc(value)</code> so the next reset uses it
0x008	64	RESET_VECTOR_1	same for hart 1
0x010	64	RESET_VECTOR_2	same for hart 2
0x018	64	RESET_VECTOR_3	same for hart 3
0x040	32	CORE_ENABLE	bit- <i>i</i> clear parks hart <i>i</i> on <code>core_enable_event_[i]</code>
0x100	32	LOCAL_BASE	RO, value = <code>cfg.local_base_default</code>
0x104	32	GLOBAL_BASE	RW, firmware-programmable
0x108	32	REGION_SIZE	RW
0x200	32	INIT_MEM_DONE	RO; driven by peer Scratchpad SRAM via <code>set_init_mem_done(bool)</code> — tested in <code>cluster_tb</code> (ScratchpadSramStub, TC-CPU-004/005)
0x204	32	DISABLE_SRAM_AUTO_INIT	RW strap; sampled by scratchpad peer on reset — tested in <code>cluster_tb</code>
0x208	32	MEM_REPAIR_STATUS	RO; flipped by <code>cluster.set_mem_repair_status(uint32_t)</code>

The decode tree is `ctrl_b_transport()` and only accepts 32- and 64-bit aligned accesses; everything else returns `TLM_GENERIC_ERROR_RESPONSE`. The `cluster_ctrl_test` GoogleTest binary covers every offset, RO/RW boundary, and the hart-reset-vector / core-enable side effects.

4.9 Temporal decoupling (`t1m_quantumkeeper`)

One `t1m_quantumkeeper` per hart (`qk_[i]`). Conventions:

- Default `quantum_insts_ = 1000` instructions per `qk.sync()` boundary; the per-instruction tick is `1 ns` (override at construction).
- `hart_thread(i)` calls `qk_[i].inc(retired × tick); if qk_[i].need_sync() qk_[i].sync();` after each `step(K)` call.
- On WFI park, the thread calls `qk_[i].sync()` immediately, **then** `wait(wfi_event_[i] | core_enable_event_[i])`, so the hart's local time is published before it goes to sleep.
- The memory-bridge callbacks read `qk.get_local_time()` for the `delay` argument of `b_transport(...)`, and store the returned delay back into the quantum keeper (`qk.set(delay)`).

This is the Bucket-A initiator pattern from `02_SMC_IP_LowLevel_Design.pdf §3.5`.

4.10 ELF loading

The cluster's constructor (or a deferred `load_elf(paths)` call) invokes `whisper_sys->loadElfFiles(paths, /*raw*/false, /*verbose*/false)`. Because the read / write memory callbacks are registered **before** `loadElfFiles()` runs, the ELF segments populate `mem_buf_` through the same fast-mem / slow-mem decision tree as run-time loads — i.e. ELF load exercises the very same address-decode and TLM extension path as a hart-issued store, so any change to that path is automatically covered by image-load tests.

`tohost` / `fromhost` symbols are resolved with `System::findElfSymbol` and stashed in `tohost_addr_` / `fromhost_addr_` for testbench polling.

5. Class & component diagrams

5.1 Class diagram (UML)

Stereotype key:

Stereotype	Meaning
<code><<sc_module>></code>	SystemC module (registered with the kernel; can have processes)
<code><<C++ interface>></code>	Pure abstract class — only virtual methods
<code><<C++ class>></code>	Plain C++ class with state and behaviour
<code><<C++ template>></code>	Templated class instantiated as <code><uint64_t></code>
<code><<SystemC>></code>	A SystemC primitive (e.g. <code>sc_event</code>)
<code><<SystemC port>></code>	A directional port that binds to a channel (<code>sc_in</code>)
<code><<TLM-2.0>></code>	TLM-2.0 utility (sockets, payload, quantum keeper)
<code><<external>></code>	Lives in another SMC IP (CLINT, PLIC, BEU, Debug Module)

Relationship key:

- `<|..` realises (interface implementation)
- `*--` composition (owns; lifetime tied)
- `o--` aggregation (references; independent lifetime)
- `..>` dependency / uses (call, no ownership)

```

classDiagram
    direction LR

    class smc_cpu_cluster {
        <<sc_module>>
        +data : simple_initiator_socket
        +mmio : simple_initiator_socket
        +ifetch : simple_initiator_socket
        +ctrl : simple_target_socket
        +irq_sw[N] : sc_in_bool
        +irq_timer[N] : sc_in_bool
        +irq_ext[N] : sc_in_bool
        -whisper_sys_ : unique_ptr_System
        -harts_[N] : iss_backend_whisper
        -qk_[N] : tlm_quantumkeeper
        -wfi_event_[N] : sc_event
        -core_enable_event_[N] : sc_event
        -mem_buf_ : vector_uint8
        -current_hart_ : unsigned
        +load_elf(paths) bool
        +hart(i) iss_hart_ref
        +num_harts() unsigned
        +inject_nmi(i, cause)
        +set_init_mem_done(b)
        +set_mem_repair_status(v)
        -hart_thread(i)
        -irq_aggregator(i)
        -mem_read_cb(addr, size, data_out) bool
        -mem_write_cb(addr, size, data_in) bool
        -tlm_access(cmd, addr, size, data) bool
        -ctrl_b_transport(trans, delay)
    }

    class iss_hart {
        <<C++ interface>>
        +reset()
        +step(K) uint32
        +set_reset_pc(addr)
        +get_pc() uint64
        +poke_mip(val)
        +read_csr(num) uint64
        +is_wfi() bool
        +clear_wfi()
        +inject_nmi(cause)
        +current_priv() uint8
        +last_commit() commit_entry
        +load_elf(paths) bool
        +mem_read(addr, size, data_out) bool
        +mem_write(addr, size, data_in) bool
    }

    class iss_backend_whisper {
        <<C++ class>>
        -sys_ : System_ref
        -hart_ : shared_ptr_Hart
        -di_ : unique_ptr_DecodedInst
    }

```

```

    -cfg_ : iss_hart_config
    -tohost_addr_ : uint64
    -fromhost_addr_ : uint64
    -wfi_active_ : bool
    -last_commit_ : commit_entry
    +reset()
    +step(K) uint32
    +poke_mip(val)
    +load_elf(paths) bool
}

class WdRiscv_System {
    <<C++ template>>
    +System(coreCount, hartsPerCore, hartIdOffset, memSize, pageSize)
    +ithHart(i) shared_ptr_Hart
    +hartCount() unsigned
    +loadElfFiles(paths, raw, verbose) bool
    +findElfSymbol(name, ElfSymbol) bool
    +defineReadMemoryCallback(fn)
    +defineWriteMemoryCallback(fn)
}

class WdRiscv_Hart {
    <<C++ template>>
    +singleStep(DecodedInst)
    +reset(resetMmRegs)
    +pokePc(addr)
    +peekPc() uint64
    +externalPokeCsr(csr, val, virtMode) bool
    +peekCsr(csr, quiet) uint64
    +configIsa(string, updateMisa) bool
    +definePmaRegion(ix, low, high, pma) bool
    +defineResetPc(addr)
    +setPendingNmi(cause)
    +privilegeMode() uint8
}

class simple_initiator_socket {
    <<TLM-2.0>>
    +operator->() interface
    +b_transport(payload, delay)
    +bind(target_socket)
}

class simple_target_socket {
    <<TLM-2.0>>
    +register_b_transport(handler)
    +bind(initiator_socket)
}

class tlm_quantumkeeper {
    <<TLM-2.0>>
    +inc(time)
    +sync()
    +get_local_time() time
    +set(time)
    +need_sync() bool
    +reset()
}

```

```

class smc_axi_extension {
  <<TLM-2.0>>
  +source_id : uint16
  +axi_id : uint16
  +prot : uint8
  +is_user : bool
  +is_fetch : bool
  +is_secure : bool
  +is_locked : bool
}

class sc_event { <<SystemC>>+notify(time) }
class sc_in_bool {
  <<SystemC port>>
  +read() bool
  +value_changed_event() event
  +bind(channel)
}

class CLINT { <<external>> }
class PLIC { <<external>> }
class BEU { <<external>> }
class Debug_Module { <<external>> }

iss_hart <|.. iss_backend_whisper : realises

smc_cpu_cluster *-- "1" WdRiscv_System : owns whisper_sys_
smc_cpu_cluster *-- "N" iss_backend_whisper : owns harts_
smc_cpu_cluster *-- "N" tlm_quantumkeeper : owns qk_
smc_cpu_cluster *-- "N" sc_event : owns wfi_event_
smc_cpu_cluster *-- "3" simple_initiator_socket : data / mmio / ifetch
smc_cpu_cluster *-- "1" simple_target_socket : ctrl
smc_cpu_cluster *-- "3N" sc_in_bool : irq_sw / irq_timer / irq_ext

iss_backend_whisper o-- WdRiscv_System : ref sys_
iss_backend_whisper o-- WdRiscv_Hart : shared_ptr hart_

WdRiscv_System o-- "N" WdRiscv_Hart : aggregates ithHart(i)

simple_initiator_socket ..> smc_axi_extension : attaches per access
sc_in_bool ..> CLINT : driven by
sc_in_bool ..> PLIC : driven by
sc_in_bool ..> BEU : driven by
sc_in_bool ..> Debug_Module : driven by

```

5.2 Component view (num_harts = 4)

```

flowchart TB
    DATA["data initiator"]
    MMIO["mmio initiator"]
    IFET["ifetch initiator"]
    CTRL["ctrl target"]

    IRQSW["irq_sw 0..3"]
    IRQTM["irq_timer 0..3"]
    IRQEX["irq_ext 0..3"]

    HT["hart_thread x4 SC_THREAD"]
    IA["irq_aggregator x4 SC_METHOD"]

    QK["tlm_quantumkeeper qk_ 0..3"]
    WEV["sc_event wfi_event_ 0..3"]
    CEV["sc_event core_enable_event_ 0..3"]
    MBUF["mem_buf_ fast-mem window"]
    CUR["current_hart_ callback context"]

    BE["iss_backend_whisper x4"]

    SYS["WdRiscv System uint64 t"]
    H0["Hart 0 mhartid 0"]
    H1["Hart 1 mhartid 1"]
    H2["Hart 2 mhartid 2"]
    H3["Hart 3 mhartid 3"]
    RCB["read callback"]
    WCB["write callback"]

    IRQSW --> IA
    IRQTM --> IA
    IRQEX --> IA

    IA -->|"externalPokeCsr MIP"| BE
    IA -->|"notify"| WEV
    HT -->|"step(K)"| BE
    HT -->|"wait on WFI"| WEV
    HT -->|"wait on disable"| CEV
    HT -->|"qk inc + sync"| QK
    CTRL -->|"CORE_ENABLE bit-i"| CEV

    BE --> H0
    BE --> H1
    BE --> H2
    BE --> H3
    H0 --> SYS
    H1 --> SYS
    H2 --> SYS
    H3 --> SYS

    SYS --> RCB
    SYS --> WCB

    RCB -->|"fast window"| MBUF

```

```

WCB -->|"fast window"| MBUF
RCB -->|"mmio path"| MMIO
WCB -->|"mmio path"| MMIO
RCB -->|"data path"| DATA
WCB -->|"data path"| DATA
RCB -->|"ifetch path"| IFET
WCB -->|"ifetch path"| IFET
CUR -->|"selects"| QK

```

5.3 Runtime sequence — MMIO load

```

sequenceDiagram
    participant HT as hart_thread
    participant BE as iss_backend_whisper
    participant W as Whisper Hart
    participant CB as mem_read_cb
    participant QK as qk
    participant FAB as mmio target

    HT->>HT: set current_hart_
    HT->>BE: step(K)
    BE->>W: singleStep(di_)
    W->>CB: readCallback(addr, size)
    CB->>CB: pick_socket(addr) → mmio
    CB->>QK: delay = get_local_time()
    CB->>CB: attach smc_axi_extension
    CB->>FAB: b_transport(TLM_READ_COMMAND)
    FAB-->>CB: TLM_OK_RESPONSE
    CB->>QK: set(delay)
    CB-->>W: return data
    W-->>BE: singleStep returns
    BE-->>HT: step returns retired count
    HT->>QK: qk.inc(retired × tick)
    HT->>QK: qk.sync() if need_sync()

```

5.4 Runtime sequence — IRQ wakes a parked hart

```
sequenceDiagram
    participant EXT as IRQ source (PLIC / CLINT)
    participant SIG as sc_signal bool
    participant IA as irq_aggregator
    participant BE as iss_backend_whisper
    participant W as Whisper Hart
    participant WEV as wfi_event
    participant HT as hart_thread (parked)

    EXT->>SIG: write(true)
    SIG->>IA: value_changed_event
    IA->>IA: snapshot parked = is_wfi()
    IA->>IA: pack MIP from 3 input bits
    IA->>BE: poke_mip(MIP)
    BE->>W: externalPokeCsr(CsrNumber::MIP, MIP)
    BE->>BE: wfi_active_ = false
    IA->>WEV: notify() if parked && MIP != 0
    WEV-->>HT: wake
    HT->>BE: step(K)
    BE->>W: singleStep - first insn after WFI
```

6. Build & dependency view

The cluster builds against:

- Accellera **SystemC 2.3.4** (built with C++20) — `SYSTEMC_HOME`
- Tenstorrent **Whisper** ≥ pinned SHA — `WHISPER_HOME` (built with `MEM_CALLBACKS=1`)
- **Boost** ≥ 1.74 — `BOOST_DIR`
- Optional: Accellera **SystemC CCI 1.0** — `CCI_HOME` (required when `SMC_BUILD_PLIC_INTEGRATION=ON`, the default; links real PLIC into `cluster_tb`)

```

flowchart LR
    WCMK["whisper-cmake (CMakeLists)"]
    RVCORE["whisper::rvcore (librvcore.a, whole-archive)"]
    VMEM["whisper virtual_memory"]
    PCI["whisper pci"]
    SF["whisper softfloat"]
    WW["whisper::whisper (INTERFACE)"]

    ISSH["iss_hart.h"]
    ISSWH["iss_backend_whisper.h"]
    ISSWC["iss_backend_whisper.cpp"]
    CLH["smc_cpu_cluster.h"]
    CLC["smc_cpu_cluster.cpp"]
    SAE["smc_axi_extension.h"]
    CLLIB["libsmc_cpu_cluster.a"]

    PLICIP["smc_plic_ip.a (peripherals/plic/src/plic.cpp)"]
    CCI["systemc::cci (CCI_HOME/lib[64])"]

    CLTB["cluster_tb (test/cluster_tb.cpp)"]

    SC["systemc::systemc (C++20)"]
    BH["Boost::boost (headers)"]
    BPO["Boost::program_options"]

    WCMK --> RVCORE
    WCMK --> VMEM
    WCMK --> PCI
    WCMK --> SF
    WCMK --> WW
    RVCORE --> WW
    VMEM --> WW
    PCI --> WW
    SF --> WW
    BH --> WW
    BPO --> WW

    ISSH --> ISSWH
    ISSWH --> ISSWC
    SAE --> CLC
    SAE --> CLLIB
    ISSWC --> CLLIB
    CLH --> CLC
    CLC --> CLLIB
    WW --> CLLIB
    SC --> CLLIB
    BH --> CLLIB

    SAE --> PLICIP
    CCI --> PLICIP
    SC --> PLICIP

    CLLIB --> CLTB
    PLICIP --> CLTB
    CCI --> CLTB

```

Key CMake hooks (top-level `CMakeLists.txt`):

```
# Required
SYSTEMC_HOME : SystemC install (include/systemc.h + lib64/libsystemc.so)
WHISPER_HOME  : Whisper source tree (build-Linux/librvcore.a)
BOOST_DIR     : Boost install (include/boost/version.hpp)

# Optional (default ON)
SMC_BUILD_PLIC_INTEGRATION : compile peripherals/plic/src/plic.cpp into
                             smc_plic_ip.a and link it into cluster_tb
CCI_HOME      : Accellera SystemC CCI 1.0 install (required when the
                             option above is ON)
```

7. Conformance with §3 of 02_SMC_IP_LowLevel_Design

The audit-closed share of 02_SMC_IP_LowLevel_Design.pdf §3 is summarised below. Items left blocked on peer IPs are tabulated in §8.

§	Design point	Status	Implementation
3.3	TLM-2.0 sockets <code>data / mmio / ifetch</code>	done	Three <code>simple_initiator_socket<...,64>;</code> <code>pick_socket()</code> routes by address (<code>mmio_lo / mmio_hi</code> vs everywhere else). <code>ifetch</code> is currently routed identically to <code>data</code> (reserved for a future ICache hook).
3.3	TLM-2.0 target socket <code>ctrl</code> (8 KiB CPU-Control regs)	done	<code>simple_target_socket<...,64></code> <code>ctrl</code> ; <code>ctrl_b_transport()</code> decodes 32 / 64-bit accesses against <code>regs_</code> (see §4.8).
3.4	Per-hart <code>iss_hart</code> + Whisper backend, <code>sc_vector</code> wiring	done	<code>sc_vector<sc_in<bool>></code> for the IRQ legs; <code>unique_ptr<iss_backend_whisper></code> array for <code>harts_</code> .
3.5	Per-hart execution loop (<code>SC_THREAD</code>), temporal decoupling	done	<code>hart_thread()</code> uses <code>step(K)</code> with <code>K = quantum_insts</code> (default 16); quantum keeper advances by <code>retired × tick</code> (≥ 1 ns).
3.5	Per-hart <code>CORE_ENABLE</code> gating	done	<code>hart_thread()</code> parks on <code>core_enable_event[i]</code> whenever bit <code>i</code> of <code>regs_.core_enable</code> is clear. Re-enable triggers <code>core_enable_event[i].notify()</code> . WFI park waits on <code>wfi_event[i]</code> or <code>core_enable_event[i]</code> .
3.6	Fast-mem fast path + TLM slow path	done	<code>mem_buf_</code> covers <code>[fast_mem_lo, fast_mem_hi)</code> ; slow path picks one of three sockets via <code>pick_socket()</code> .
3.7	Per-hart IRQ inputs (<code>msip / mtip / meip</code>)	done	<code>irq_aggregator()</code> packs them into MIP, calls <code>poke_mip()</code> , and notifies <code>wfi_event[i]</code> when a parked hart needs to wake.
3.8	CPU-Control register file at BASE + <code>0x001_0000</code>	done	See §4.8 for the full register map.
3.8	<code>RESET_VECTOR_N[i]</code> write → <code>hart(i).set_reset_pc()</code>	done	Inside <code>ctrl_b_transport()</code> .
3.8	<code>INIT_MEM_DONE / MEM_REPAIR_STATUS</code> testbench hooks	done	<code>set_init_mem_done()</code> exercised by <code>ScratchpadSramStub</code> in <code>cluster_tb</code> (TC-CPU-004/005). Production scratchpad TLM on <code>data</code> still open.
3.9	Debug API: <code>step(K)</code> , <code>inject_nmi</code> , <code>last_commit()</code>	done	<code>iss_hart::step(unsigned n)</code> , <code>iss_hart::inject_nmi(cause)</code> (<code>Hart::setPendingNmi</code>), <code>iss_hart::last_commit()</code> populated each <code>step()</code> . <code>cluster.inject_nmi(i, cause)</code> wakes a parked hart.
3.9	<code>iss_hart::current_priv()</code>	done	Live <code>Hart::privilegeMode()</code> (0 = U, 1 = S, 3 = M); drives <code>prot[2]</code> .
3.10	<code>smc_axi_extension</code> on every outgoing transaction	done	<code>libsmc/cpu/smc_axi_extension.h</code> (§4.7); <code>tlm_access()</code> allocates and attaches one extension per GP; ownership transferred to the GP.
3.10	<code>prot[3]</code> (lock) for AMO / LR-SC	done	<code>cfg.amo_lock_detect == true</code> : <code>tlm_access()</code> inspects <code>iss_hart::last_commit().opcode</code> (AMO opcode = <code>0x2F</code>).
3.10	<code>prot[2]</code> (privilege) for U vs M/S	done	<code>tlm_access()</code> reads <code>iss_hart::current_priv()</code> .
3.10	Unified <code>smc_axi_extension</code> shared with peer IPs (ODR-safe)	done	PLIC's <code>smc_tlm_extensions.h</code> is a thin shim that <code>__has_include s</code> <code>smc_axi_extension.h</code> and delegates to the cluster's canonical class. Exercised when <code>cluster_tb</code> links <code>smc_plic_ip</code> .

8. Peer-IP integration status

Detail by individual IP is in `03_CPU_Cluster_Test_Plan.md §13`. This table is the chiplet-level summary.

PLIC vs CLINT in verification: the full **PLIC IP** matrix lives in `peripherals/plic/test/plic_tb`. `cluster_tb` proves **cluster** ↔ **PLIC** (CPU programs PLIC over `mmio`, `irq_ext` from `ctx_out`, ISR claim/complete). The **CLINT IP** is not modeled here; `cluster_tb` only toggles `irq_sw` / `irq_timer` as wire-level stubs (§8.6 of the test plan).

Peer IP / line	Cluster-side hook	Status
<code>CLINT.msip_out[i]</code>	<code>irq_sw[i]</code> (<code>sc_in<bool></code>)	Cluster side ready; CLINT IP + MMIO out of scope for <code>cluster_tb</code> (wire stubs §8.6)
<code>CLINT.mtip_out[i]</code>	<code>irq_timer[i]</code> (<code>sc_in<bool></code>)	Cluster side ready; CLINT IP + MMIO out of scope for <code>cluster_tb</code> (wire stubs §8.6)
<code>PLIC.ctx_out[2*h + 0]</code>	<code>irq_ext[i]</code> (<code>sc_in<bool></code>)	Integration green (<code>cluster_tb</code> §8.7): CPU MMIO configures PLIC; TB drives <code>src_in</code> ; firmware ISR claim/complete. Full PLIC matrix: <code>plc_tb</code> . S-mode context open.
<code>PLIC.ctx_out[2*h + 1]</code>	<code>irq_ext_s[i]</code> (planned port)	Open — see <code>03_CPU_Cluster_Test_Plan.md §13.1</code>
<code>PLIC.seip / claim-complete</code>	shared with <code>irq_ext</code> + <code>mmio</code>	Open — folded into the S-mode leg above
<code>BEU.nmi_out[i]</code>	<code>beu_nmi_in[i]</code> (planned port; trivial to add)	Cluster has <code>iss_hart::inject_nmi(cause)</code> ready; awaiting BEU IP and the port
Debug Module → cluster	<code>debug_irq_in</code> (planned)	Awaiting Debug Module IP
Reset Unit → cluster	reset network (Cold / Cool / Core / FLR / WDT)	Awaiting Reset Unit IP; cluster currently consumes a single boolean <code>rst_n_i</code>
Boot ROM + Scratchpad SRAM	<code>data / ifetch</code> initiator sockets	Init handshake via <code>ScratchpadSramStub</code> in <code>cluster_tb</code> ; full scratchpad map + Boot ROM still open

The cluster's interfaces are **strictly additive** from this list — any future peer IP plugs into either an existing socket or a held-low IRQ input, with no changes to the cluster's own source.

9. References

- `01_SMC_Architecture.pdf` — chiplet-level architecture (§1 purpose, §2 top-level, §3 clock/reset, §4 fabric, §5 IP catalogue, §6 modelling conventions, §7 SMU integration).
- `02_SMC_IP_LowLevel_Design.pdf §3` — CPU cluster low-level design (3.3 sockets, 3.5 quantum keeper, 3.7 IRQ aggregator, 3.8 CPU Control, 3.9 debug API, 3.10 sideband extension).
- `03_SMC_Test_Plan.pdf §A.1 – A.6` and `§B.3` — verification methodology and TC-CPU-001 .. 010 catalogue.
- `03_CPU_Cluster_Test_Plan.md` — markdown test plan for this IP, including the §13.1 PLIC-integration status table.
- `04_CCI_Integration_Guide.md` — adopting SystemC CCI for the cluster's `config` parameters and how peer IPs (e.g. PLIC) expose `cci_params`.

- `peripherals/plic/README.md` — sibling IP (`plic_tb` = full PLIC matrix; `cluster_tb` = cluster ↔ PLIC integration only).

End of document.